

6 - Local Search and Hill Climbing

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Explain how local search differs from path-based search.
- Implement hill climbing for an optimization problem.
- Identify local maxima, plateaus, and ridges and test simple remedies.

Background

Local search is useful when the final configuration matters more than the path used to reach it. It keeps little history and can handle very large spaces, but greedy improvement can become trapped in locally good regions.

Core Concepts

Concept	Meaning in this lab
Objective function	Scores a candidate solution.
Neighbor	A solution reachable by a small modification.
Hill climbing	Moves to a better neighbor until no improvement is available.
Local optimum	A solution better than its neighbors but not necessarily globally best.
Random restart	Repeats local search from multiple initial solutions.

Guided Procedure

1. Choose a representation for the 8-Queens problem: one queen per column, with row positions stored in a list.
2. Define a conflict count; lower is better.
3. Generate neighbors by moving one queen within its column.
4. Move to the neighbor with the fewest conflicts when it improves the score.
5. Add random restarts and compare the success rate.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import random

def conflicts(board):
    c = 0
    for i in range(len(board)):
        for j in range(i + 1, len(board)):
            same_row = board[i] == board[j]
            same_diag = abs(board[i] - board[j]) == abs(i - j)
            c += same_row or same_diag
    return c

def random_board(n=8):
    return [random.randrange(n) for _ in range(n)]
```

Lab Exercises

Task 1. Complete steepest-ascent hill climbing for 8-Queens.

Task 2. Measure success over 50 random starts.

Task 3. Implement sideways moves with a fixed limit and compare results.

Task 4. Design a hill-climbing solution for a different problem such as feature subset selection or simple scheduling.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why does hill climbing not need a search tree?
2. What is a plateau?
3. How do random restarts improve robustness?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.